

Entrenamiento de Red Neuronal con Keras en “R”

Guillermo Cardenas, Juan Suarez

Link del colab: https://colab.research.google.com/drive/1sj6da-qZM_h3ieu2Bvqw6njyanliQ6Jw?usp=sharing

Introducción

Este reporte muestra el uso de **R** para entrenar un modelo de red neuronal utilizando los paquetes Keras y TensorFlow.

Configuración Inicial

Instalación de Paquetes

```
if (!requireNamespace("tensorflow", quietly = TRUE))
install.packages("tensorflow")
if (!requireNamespace("keras", quietly = TRUE)) install.packages("keras")
library(keras)
library(tensorflow)
library(ggplot2)
library(gridExtra)
library(png)
```

Configuración inicial del dataset

```
dataset_path <- "C:/Users/aasal/OneDrive/Documentos/Curso
Samsung/.venv/IA uni/Alfabeto/dataset400"
epochs <- as.integer(10)
batch_size <- as.integer(32)
target_size <- c(32, 32)
classes <- c("A", "B", "C", "D", "E", "F", "G", "H", "I", "K", "L", "M",
"N", "O", "P", "Q", "R", "S", "T", "U", "V", "W", "X", "Y")
```

#Preprocesamiento de Datos

```
train_datagen <- image_data_generator(
  rescale = 1 / 255,
  rotation_range = 20,
  width_shift_range = 0.2,
  height_shift_range = 0.2,
  horizontal_flip = TRUE,
```

```

    validation_split = 0.2
)

train_generator <- flow_images_from_directory(
  directory = dataset_path,
  generator = train_datagen,
  target_size = target_size,
  color_mode = "grayscale",
  batch_size = batch_size,
  class_mode = "categorical",
  classes = classes,
  subset = "training",
  seed = 123
)

## Found 7680 images belonging to 24 classes.

validation_generator <- flow_images_from_directory(
  directory = dataset_path,
  generator = train_datagen,
  target_size = target_size,
  color_mode = "grayscale",
  batch_size = batch_size,
  class_mode = "categorical",
  classes = classes,
  subset = "validation",
  seed = 123
)

## Found 1920 images belonging to 24 classes.

```

Definición del Modelo CNN

```

# Definición de la arquitectura
inputs <- layer_input(shape = c(32, 32, 1))

outputs <- inputs %>%

  layer_conv_2d(filters = 64, kernel_size = c(3,3), activation = "relu",
padding = "same") %>%
  layer_conv_2d(filters = 64, kernel_size = c(3,3), activation = "relu",
padding = "same") %>%
  layer_batch_normalization() %>%
  layer_max_pooling_2d(pool_size = c(2,2)) %>%
  layer_dropout(rate = 0.25) %>%

  layer_conv_2d(filters = 128, kernel_size = c(3,3), activation = "relu",
padding = "same") %>%
  layer_conv_2d(filters = 128, kernel_size = c(3,3), activation = "relu",
padding = "same") %>%

```

```

layer_batch_normalization() %>%
layer_max_pooling_2d(pool_size = c(2,2)) %>%
layer_dropout(rate = 0.25) %>%

layer_flatten() %>%
layer_dense(units = 256, activation = "relu") %>%
layer_dense(units = 64, activation = "relu") %>%
layer_dropout(rate = 0.3) %>%
layer_dense(units = 24, activation = "softmax")

model <- keras_model(inputs = inputs, outputs = outputs)

```

Compilacion del modelo

```

# Resumen del modelo
summary(model)

## <Functional name=functional, built=True>

```

Callbacks necesarios para salvar

```

# Callbacks
checkpoint_dir <- "C:/Users/aasal/OneDrive/Documentos/Curso
Samsung/.venv/IA uni/Alfabeto/save"
if (!dir.exists(checkpoint_dir)) {
  dir.create(checkpoint_dir, recursive = TRUE)
}

checkpoint_callback <- callback_model_checkpoint(
  filepath = file.path(checkpoint_dir, "weights-{epoch:02d}-
{val_accuracy:.2f}.h5"), # Cambiado a .h5
  monitor = "val_accuracy",
  save_best_only = TRUE,
  mode = "max",
  verbose = 1
)

early_stop <- callback_early_stopping(
  monitor = "val_loss",
  patience = 5,
  restore_best_weights = TRUE
)

```

Entrenamiento del modelo

```

# 3. Compilar el modelo
model$compile(
  optimizer = optimizer_adam(),

```

```

    loss = "categorical_crossentropy", # Cambiado a
categorical_crossentropy
    metrics = list("accuracy")
)

# Resumen del modelo
summary(model)
# 4. Entrenar el modelo
history <- model$fit(
    train_generator,
    epochs = epochs, # Usar la variable definida
    validation_data = validation_generator,
    callbacks = list(checkpoint_callback, early_stop),
    verbose = 1
)

# 5. Evaluar el modelo
evaluation <- model$evaluate(validation_generator)
cat(sprintf("Pérdida en validación: %.4f\n", evaluation["loss"]))
cat(sprintf("Precisión en validación: %.4f\n",
evaluation["accuracy"]))

```

Found 7680 images belonging to 24 classes.

Found 1920 images belonging to 24 classes.

<Functional name=functional_3, built=True>

Epoch 1/10

240/240 ————— **19s** 70ms/step - accuracy: 0.0510 - loss: 3.1517

Epoch 1: val_accuracy improved from -inf to 0.12500, saving model to

C:/Users/aasal/OneDrive/Documentos/Curso Samsung/.venv/IA uni/Alfabeto/save/weights-01-0.12.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

240/240 ————— **22s** 81ms/step - accuracy: 0.0511 - loss: 3.1514 - val_accuracy: 0.1250 - val_loss: 2.9117

Epoch 2/10

240/240 ————— **16s** 64ms/step - accuracy: 0.1838 - loss: 2.6366

Epoch 2: val_accuracy improved from 0.12500 to 0.22031, saving model to

C:/Users/aasal/OneDrive/Documentos/Curso Samsung/.venv/IA uni/Alfabeto/save/weights-02-0.22.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

240/240 ————— 18s 74ms/step - accuracy: 0.1839 - loss: 2.6360 - val_accuracy: 0.2203 - val_loss: 2.5516
Epoch 3/10

240/240 ————— 16s 65ms/step - accuracy: 0.3016 - loss: 2.2003

Epoch 3: val_accuracy improved from 0.22031 to 0.25885, saving model to C:/Users/aasal/OneDrive/Documentos/Curso Samsung/.venv/IA uni/Alfabeto/save/weights-03-0.26.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

240/240 ————— 19s 78ms/step - accuracy: 0.3017 - loss: 2.2001 - val_accuracy: 0.2589 - val_loss: 2.4173
Epoch 4/10

240/240 ————— 15s 64ms/step - accuracy: 0.3822 - loss: 1.9369

Epoch 4: val_accuracy improved from 0.25885 to 0.29115, saving model to C:/Users/aasal/OneDrive/Documentos/Curso Samsung/.venv/IA uni/Alfabeto/save/weights-04-0.29.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

240/240 ————— 18s 74ms/step - accuracy: 0.3823 - loss: 1.9367 - val_accuracy: 0.2911 - val_loss: 2.2943
Epoch 5/10

240/240 ————— 16s 65ms/step - accuracy: 0.4493 - loss: 1.7054

Epoch 5: val_accuracy did not improve from 0.29115

240/240 ————— 18s 74ms/step - accuracy: 0.4493 - loss: 1.7054 - val_accuracy: 0.2833 - val_loss: 2.2434
Epoch 6/10

240/240 ————— 17s 71ms/step - accuracy: 0.4861 - loss: 1.5901

Epoch 6: val_accuracy did not improve from 0.29115

240/240 ————— 19s 80ms/step - accuracy: 0.4862 - loss: 1.5901 - val_accuracy: 0.2786 - val_loss: 2.2225
Epoch 7/10

240/240 ————— 16s 65ms/step - accuracy: 0.5042 - loss: 1.5329

Epoch 7: val_accuracy improved from 0.29115 to 0.34896, saving model to C:/Users/aasal/OneDrive/Documentos/Curso Samsung/.venv/IA uni/Alfabeto/save/weights-07-0.35.h5

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

```

240/240 ————— 18s 75ms/step - accuracy: 0.5043 - loss:
1.5326 - val_accuracy: 0.3490 - val_loss: 2.0472
Epoch 8/10
240/240 ————— 16s 66ms/step - accuracy: 0.5423 - loss:
1.3801

Epoch 8: val_accuracy improved from 0.34896 to 0.37656, saving model to
C:/Users/aasal/OneDrive/Documentos/Curso Samsung/.venv/IA uni/Alfabeto/save/weights-08-
0.38.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead
the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.
240/240 ————— 19s 79ms/step - accuracy: 0.5424 - loss:
1.3800 - val_accuracy: 0.3766 - val_loss: 1.9332
Epoch 9/10
240/240 ————— 17s 66ms/step - accuracy: 0.5797 - loss:
1.2877

Epoch 9: val_accuracy improved from 0.37656 to 0.41094, saving model to
C:/Users/aasal/OneDrive/Documentos/Curso Samsung/.venv/IA uni/Alfabeto/save/weights-09-
0.41.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead
the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.
240/240 ————— 20s 76ms/step - accuracy: 0.5797 - loss:
1.2876 - val_accuracy: 0.4109 - val_loss: 1.8455
Epoch 10/10
240/240 ————— 16s 67ms/step - accuracy: 0.5950 - loss:
1.2392

Epoch 10: val_accuracy did not improve from 0.41094
240/240 ————— 19s 80ms/step - accuracy: 0.5950 - loss:
1.2392 - val_accuracy: 0.3995 - val_loss: 1.8910
60/60 ————— 2s 39ms/step - accuracy: 0.3895 - loss: 1.8559
60/60 ————— 2s 39ms/step - accuracy: 0.3896 - loss: 1.8565
Pérdida en validación: NA
Precisión en validación: NA
WARNING:absl:The `save_format` argument is deprecated in Keras 3. We recommend removing this
argument as it can be inferred from the file path. Received: save_format=h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
`keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead
the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model,
'my_model.keras')`.
List of 2
$: num [1:32, 1:32, 1:32, 1] 0.557 0.204 0.538 0.269 0.082 ...
$: num [1:32, 1:24] 0 0 0 0 0 0 0 0 0 ...
NULL

```

Evaluar y salvar el modelo

```
# 6. Guardar el modelo
save_model_hdf5(model, "modelo_cajas.h5")
(str(generator_next(train_generator)))

## List of 2
## $ : num [1:32, 1:32, 1:32, 1] 0.149 0.708 0.788 0.238 0.066 ...
## $ : num [1:32, 1:24] 0 0 1 0 0 0 0 0 0 0 ...

## NULL
```

Graficas:

Grafica de arreglo de la red:

```
# Función para visualizar la arquitectura del modelo
plot_model_architecture <- function() {
  # Capas representadas en el gráfico
  layers <- c(
    "Input (32x32x1)",
    "Conv2D (64 filtros, ReLU, padding=same)",
    "Conv2D (64 filtros, ReLU, padding=same)",
    "BatchNorm",
    "MaxPooling2D",
    "Dropout (rate=0.25)",
    "Conv2D (128 filtros, ReLU, padding=same)",
    "Conv2D (128 filtros, ReLU, padding=same)",
    "BatchNorm",
    "MaxPooling2D",
    "Dropout (rate=0.25)",
    "Flatten",
    "Dense (256 unidades, ReLU)",
    "Dense (64 unidades, ReLU)",
    "Dropout (rate=0.3)",
    "Dense (24 unidades, Softmax)"
  )

  # Crear un data frame para las posiciones de las capas
  df <- data.frame(
    x = rep(1, length(layers)),
    y = seq(length(layers), 1),
    label = layers
  )

  # Generar el gráfico utilizando ggplot2
  ggplot(df, aes(x, y, label = label)) +
    geom_point(size = 10, color = "steelblue") + # Puntos azules que
representan las capas
    geom_text(color = "black", size = 3, fontface = "bold", vjust = -0.6)
```

```

+ # Texto en negro
  geom_segment(aes(x = x, xend = x, y = y - 0.4, yend = y + 0.4),
               arrow = arrow(length = unit(0.2, "cm")),
               color = "gray") +
  theme_void() +
  labs(title = "Arquitectura de la Red Neuronal") +
  theme(plot.title = element_text(hjust = 0.5, size = 16, face =
"bold"))
}

# Llamar a la función para visualizar el gráfico
plot_model_architecture()

```

Grafica de imagenes de ejemplo:

```

# Visualizar algunas imágenes de ejemplo
plot_sample_images <- function(generator, n = 6) {
  batch <- generator_next(generator)
  images <- batch[[1]]
  labels <- batch[[2]]

  # Configurar área de gráfico
  par(mfrow = c(2, 3), mar = c(1, 1, 2, 1))

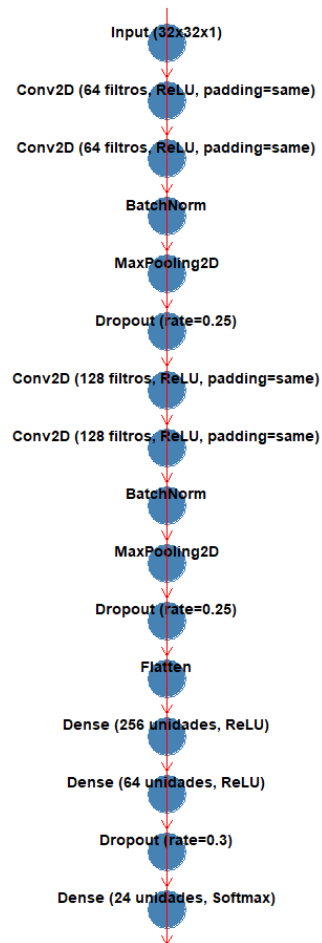
  for (i in 1:n) {
    # Extraer y normalizar la imagen (convertir a rango [0,1])
    img <- images[i,,,1]
    img_norm <- (img - min(img)) / (max(img) - min(img))

    # Crear plot con funciones base de R
    plot(NA, xlim = c(0, 1), ylim = c(0, 1), xaxt = 'n', yaxt = 'n', ann
= FALSE)
    rasterImage(img_norm, 0, 0, 1, 1)
    title(main = ifelse(labels[i] == 1, classes[2], classes[1]), line =
0.5)
    box()
  }
}

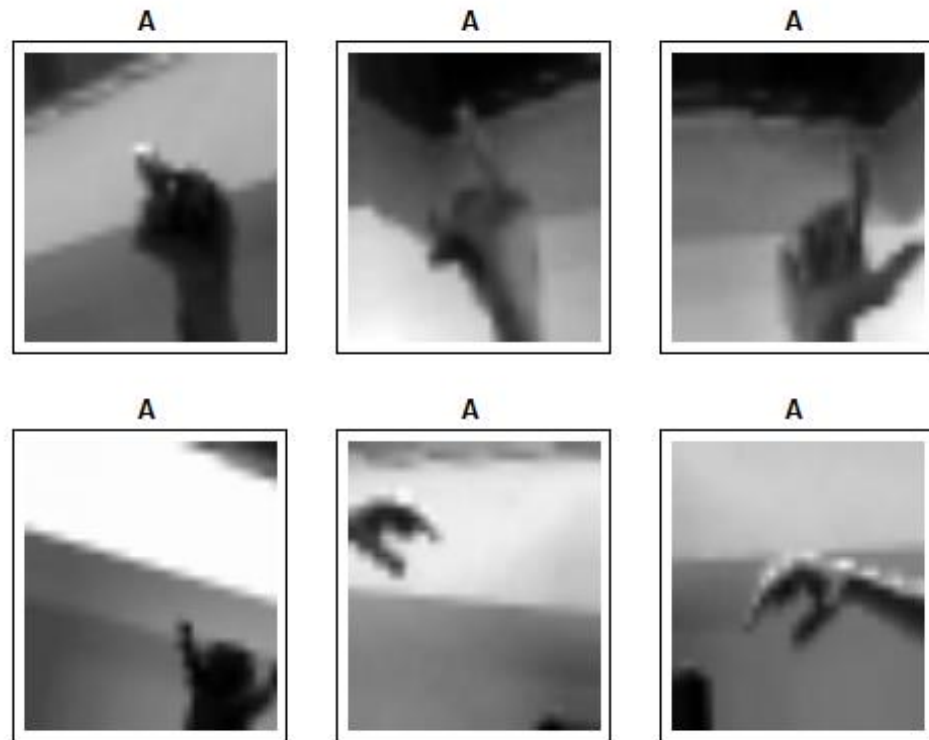
# Mostrar todas las visualizaciones
plot_model_architecture()

```

Arquitectura de la Red Neuronal



```
plot_sample_images(train_generator)
```



Grafica de entrenamiento:

```

}

# Gráficos de entrenamiento
plot_training_history <- function(history) {
  # Extraer métricas de la nueva estructura
  metrics <- history$history

  # Crear dataframe para ggplot
  df <- data.frame(
    epoch = rep(1:length(metrics$loss), 2),
    value = c(metrics$loss, metrics$val_loss,
              metrics$accuracy, metrics$val_accuracy),
    metric = rep(c("Loss", "Accuracy"), each = length(metrics$loss)),
    dataset = rep(rep(c("Train", "Validation"), each =
length(metrics$loss)), 2)
  )

  # Gráfico de pérdida
  p_loss <- ggplot(subset(df, metric == "Loss"), aes(epoch, value,
color = dataset)) +

```

```

    geom_line() + geom_point() +
    labs(title = "Training and Validation Loss", y = "Loss") +
    theme_minimal()

# Gráfico de precisión
p_acc <- ggplot(subset(df, metric == "Accuracy"), aes(epoch, value,
color = dataset)) +
    geom_line() + geom_point() +
    labs(title = "Training and Validation Accuracy", y = "Accuracy") +
    theme_minimal()

grid.arrange(p_loss, p_acc, ncol = 1)
}

plot_training_history(history)

```



VERSION DEL MODELO EN PHYTON:

Link del colab:

https://colab.research.google.com/drive/17E3Ozxa_7kIT_RDpvUAX4pV4VzWo0x6M?usp=sharing

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("grassknotted/asl-alphabet")

print("Path to dataset files:", path)
train_path = path + "/asl_alphabet_train/asl_alphabet_train"
test_path = path + "/asl_alphabet_test/asl_alphabet_test"
print("Path to train files:", train_path)
print("Path to test files:", test_path)
Downloading from https://www.kaggle.com/api/v1/datasets/download/grassknotted/asl-alphabet?dataset\_version\_number=1...
100%|██████████| 1.03G/1.03G [00:20<00:00, 54.4MB/s]
Extracting files...

Path to dataset files: /root/.cache/kagglehub/datasets/grassknotted/asl-alphabet/versions/1
Path to train files: /root/.cache/kagglehub/datasets/grassknotted/asl-alphabet/versions/1/asl_alphabet_train/asl_alphabet_train
Path to test files: /root/.cache/kagglehub/datasets/grassknotted/asl-alphabet/versions/1/asl_alphabet_test/asl_alphabet_test
```

```
clases =
["A", "B", "C", "D", "E", "F", "G", "H", "I", "K", "L", "M", "N", "O", "P", "Q", "R", "S", "T", "U", "V", "W", "X", "Y"]
len(clases)
24
```

```
import os
ruta = train_path
carpetas = os.listdir(ruta)
carpetas.sort()
for carpeta in carpetas:
    print(carpeta)
```

A
B
C
D
E
F

G
H
I
J
K
L
M
N
O
P
Q
R
S
T
U
V
W
X
Y
Z
del
nothing
space

```
import os
def length_of_folders(path, classes):
    lista = []
    for clase in classes:
        ruta = path + '/' + clase
        imagenes = os.listdir(ruta)
        tupla = (len(imagenes),clase)
        lista.append(tupla)
    lista.sort()
    for tupla in lista:
        print(f"{tupla[0]}: {tupla[1]}")
    return min(lista)[0]
train_len = length_of_folders(train_path, classes)
print(train_len)
```

3000: A
3000: B
3000: C
3000: D
3000: E
3000: F
3000: G
3000: H
3000: I
3000: K

```
3000: L
3000: M
3000: N
3000: O
3000: P
3000: Q
3000: R
3000: S
3000: T
3000: U
3000: V
3000: W
3000: X
3000: Y
3000:
```

```
import shutil
import os
import cv2
import numpy as np

def crear_data_grayscale(carpeta_fuente, carpeta_destino, num):
    imagenes = os.listdir(carpeta_fuente)
    for i, nombreimg in enumerate(imagenes):
        if i < num:
            ruta = carpeta_fuente + '/' + nombreimg
            image = cv2.imread(ruta)
            if image is None:
                print("Error: No se pudo cargar la imagen.")
                continue
            gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

            if not os.path.exists(carpeta_destino):
                os.makedirs(carpeta_destino)
            cv2.imwrite(carpeta_destino + '/' + nombreimg, gray_image)
    imagenes_salidas = os.listdir(carpeta_destino)
    print(len(imagenes_salidas))
```

```
import shutil
import os

def crear_data(carpeta_fuente, carpeta_destino, num):
    imagenes = os.listdir(carpeta_fuente)
    if not os.path.exists(carpeta_destino):
        os.makedirs(carpeta_destino)
    for i, nombreimg in enumerate(imagenes):
        if i < num:
```

```
    #Copia de la carpeta fuente a la destino
    shutil.copy(carpeta_fuente + '/' + nombreimg, carpeta_destino +
'/' + nombreimg)
    imagenes_salidas = os.listdir(carpeta_destino)
    print(len(imagenes_salidas))
```

```
def usar_funcion_de_carpetas(carpeta_fuente, carpeta_destino, clases,
funcion, num):
    for clase in clases:
        fuente = carpeta_fuente + '/' + clase
        destino = carpeta_destino + '/' + clase
        funcion(fuente, destino, num)
```

```
color = "grayscale"
```

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.layers import Dense, Conv2D, MaxPooling2D,
Flatten, Dropout
from tensorflow.keras.models import Sequential
from tensorflow.keras.optimizers import Adamax
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
from tensorflow.keras.layers import BatchNormalization
from tqdm import tqdm
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from PIL import Image
import warnings
warnings.filterwarnings('ignore')

# 1. Configurar rutas y parámetros
dataset_path = "/content/train"
target_size = (64, 64) # Redimensionar a 256x256
batch_size = 32
epochs = 20
saved_models = "/content/saved_models"

# 2. Preprocesamiento y aumento de datos
```

```

datagen = ImageDataGenerator(
    rescale = 1.0/255.0, # Normalización de píxeles
    rotation_range = 20, # Rotación aleatoria
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip = True,
    validation_split=0.3
)

# 3. Cargar las imágenes de las 4 clases
train_gen = datagen.flow_from_directory(
    dataset_path,
    target_size = target_size,
    batch_size = batch_size,
    color_mode = color,
    class_mode = 'categorical', # Clasificación multiclase
    classes = classes,
    subset='training' # Conjunto de entrenamiento
)

valid_gen = datagen.flow_from_directory(
    dataset_path,
    target_size = target_size,
    batch_size = batch_size,
    color_mode = color,
    class_mode = 'categorical', # Clasificación multiclase
    classes = classes,
    subset='validation'
)

# 4. Construir el modelo
Model = Sequential([
    Conv2D(64, kernel_size= (3,3),padding='same', activation='relu',
input_shape=(target_size[0], target_size[1],1)),
    Conv2D(64, kernel_size= (3,3),padding='same', activation='relu'),
    BatchNormalization(),
    MaxPooling2D((2,2)),

    Conv2D(128, kernel_size= (3,3),padding='same', activation='relu'),
    Conv2D(128, kernel_size= (3,3),padding='same', activation='relu'),
    BatchNormalization(),
    MaxPooling2D((2,2)),

    Conv2D(128, kernel_size= (3,3),padding='same', activation='relu'),

```

```

Conv2D(128, kernel_size= (3,3),padding='same', activation='relu'),
BatchNormalization(),
MaxPooling2D((2,2)),

Conv2D(256, kernel_size= (3,3),padding='same', activation='relu'),
Conv2D(256, kernel_size= (3,3),padding='same', activation='relu'),
BatchNormalization(),
MaxPooling2D((2,2)),

Conv2D(512, kernel_size= (3,3),padding='same', activation='relu'),
Conv2D(512, kernel_size= (3,3),padding='same', activation='relu'),
BatchNormalization(),
MaxPooling2D((2,2)),

Conv2D(512, kernel_size= (3,3),padding='same', activation='relu'),
Conv2D(512, kernel_size= (3,3),padding='same', activation='relu'),
BatchNormalization(),
MaxPooling2D((2,2)),

Flatten(),
Dense(256, activation='relu'),
Dense(64, activation='relu'),
Dropout(0.3),
Dense(24, activation='softmax')
])

# 5. Compilar el modelo
Model.compile(optimizer = "adam",
              loss='categorical_crossentropy',
              metrics=['accuracy']
              )

# 6. Resumen del modelo
Model.summary()
if not os.path.exists(saved_models):
    os.makedirs(saved_models)
filepath= "/content/saved_models/weights-improvement-{epoch:02d}-{val_accuracy:.2f}.keras" #File name includes epoch and validation accuracy.
#Use Mode = max for accuracy and min for loss.
checkpoint = ModelCheckpoint(filepath, monitor = 'val_accuracy',
verbose = 1, save_best_only = True, mode = 'max')
# 7. Callback para EarlyStopping
early_stop = EarlyStopping(

```

```

        monitor = 'val_loss', # Monitorear la pérdida en el conjunto de
validación
        patience = 5, # Número de épocas sin mejora antes de detener el
entrenamiento
        restore_best_weights=True # Restaurar los mejores pesos
encontrados
    )

callbacks_list = [checkpoint, early_stop]

# 8. Entrenar el modelo con EarlyStopping
history = Model.fit(
    train_gen,
    epochs = epochs,
    validation_data=valid_gen,
    callbacks=callbacks_list,
    verbose=1
)

# 9. Evaluar el modelo
loss, accuracy = Model.evaluate(valid_gen)
print(f"Pérdida en validación: {loss}")
print(f"Precisión en validación: {accuracy}")

# 10. Guardar el modelo
Model.save("senas_modelo.keras")

```

Found 50400 images belonging to 24 classes.

Found 21600 images belonging to 24 classes.

Model: "sequential_7"

Layer (type) Param #	Output Shape
conv2d_84 (Conv2D) 640	(None, 64, 64, 64)
conv2d_85 (Conv2D) 36,928	(None, 64, 64, 64)
batch_normalization_42 256 (BatchNormalization)	(None, 64, 64, 64)

0	max_pooling2d_41 (MaxPooling2D)	(None, 32, 32, 64)	
73,856	conv2d_86 (Conv2D)	(None, 32, 32, 128)	
147,584	conv2d_87 (Conv2D)	(None, 32, 32, 128)	
512	batch_normalization_43 (BatchNormalization)	(None, 32, 32, 128)	
0	max_pooling2d_42 (MaxPooling2D)	(None, 16, 16, 128)	
147,584	conv2d_88 (Conv2D)	(None, 16, 16, 128)	
147,584	conv2d_89 (Conv2D)	(None, 16, 16, 128)	
512	batch_normalization_44 (BatchNormalization)	(None, 16, 16, 128)	
0	max_pooling2d_43 (MaxPooling2D)	(None, 8, 8, 128)	
295,168	conv2d_90 (Conv2D)	(None, 8, 8, 256)	
590,080	conv2d_91 (Conv2D)	(None, 8, 8, 256)	
1,024	batch_normalization_45 (BatchNormalization)	(None, 8, 8, 256)	

0	max_pooling2d_44 (MaxPooling2D)	(None, 4, 4, 256)	
	conv2d_92 (Conv2D)	(None, 4, 4, 512)	
1,180,160			
	conv2d_93 (Conv2D)	(None, 4, 4, 512)	
2,359,808			
	batch_normalization_46 (BatchNormalization)	(None, 4, 4, 512)	
2,048			
	max_pooling2d_45 (MaxPooling2D)	(None, 2, 2, 512)	
0			
	conv2d_94 (Conv2D)	(None, 2, 2, 512)	
2,359,808			
	conv2d_95 (Conv2D)	(None, 2, 2, 512)	
2,359,808			
	batch_normalization_47 (BatchNormalization)	(None, 2, 2, 512)	
2,048			
	max_pooling2d_46 (MaxPooling2D)	(None, 1, 1, 512)	
0			
	flatten_7 (Flatten)	(None, 512)	
0			
	dense_21 (Dense)	(None, 256)	
131,328			
	dense_22 (Dense)	(None, 64)	
16,448			
	dropout_7 (Dropout)	(None, 64)	
0			

dense_23 (Dense)	(None, 24)
1,560	

Total params: 9,854,744 (37.59 MB)
Trainable params: 9,851,544 (37.58 MB)
Non-trainable params: 3,200 (12.50 KB)

Epoch 1/20

1575/1575 ————— 0s 48ms/step - accuracy: 0.0456 - loss: 3.1951
 Epoch 1: val_accuracy improved from -inf to 0.04162, saving model to /content/saved_models/weights-improvement-01-0.04.keras

1575/1575 ————— 120s 67ms/step - accuracy: 0.0456 - loss: 3.1951 - val_accuracy: 0.0416 - val_loss: 3.1785

Epoch 2/20

1574/1575 ————— 0s 47ms/step - accuracy: 0.0664 - loss: 3.0206
 Epoch 2: val_accuracy improved from 0.04162 to 0.11444, saving model to /content/saved_models/weights-improvement-02-0.11.keras

1575/1575 ————— 100s 64ms/step - accuracy: 0.0664 - loss: 3.0205 - val_accuracy: 0.1144 - val_loss: 3.0117

Epoch 3/20

1575/1575 ————— 0s 48ms/step - accuracy: 0.1154 - loss: 2.8389
 Epoch 3: val_accuracy did not improve from 0.11444

1575/1575 ————— 143s 64ms/step - accuracy: 0.1154 - loss: 2.8388 - val_accuracy: 0.0824 - val_loss: 4.5897

Epoch 4/20

1574/1575 ————— 0s 48ms/step - accuracy: 0.2255 - loss: 2.3440
 Epoch 4: val_accuracy improved from 0.11444 to 0.18671, saving model to /content/saved_models/weights-improvement-04-0.19.keras

1575/1575 ————— 101s 64ms/step - accuracy: 0.2255 - loss: 2.3438 - val_accuracy: 0.1867 - val_loss: 3.0747

Epoch 5/20

1575/1575 ————— 0s 48ms/step - accuracy: 0.3269 - loss: 2.0167
 Epoch 5: val_accuracy improved from 0.18671 to 0.24722, saving model to /content/saved_models/weights-improvement-05-0.25.keras

1575/1575 ————— 142s 64ms/step - accuracy: 0.3269 - loss: 2.0166 - val_accuracy: 0.2472 - val_loss: 3.0193

Epoch 6/20

1574/1575 ————— 0s 50ms/step - accuracy: 0.4265 - loss: 1.7078
 Epoch 6: val_accuracy improved from 0.24722 to 0.32130, saving model to /content/saved_models/weights-improvement-06-0.32.keras

1575/1575 ————— 145s 66ms/step - accuracy: 0.4265 - loss: 1.7077 - val_accuracy: 0.3213 - val_loss: 2.7410

Epoch 7/20

1575/1575 ————— 0s 48ms/step - accuracy: 0.5663 - loss: 1.2805
 Epoch 7: val_accuracy improved from 0.32130 to 0.37032, saving model to /content/saved_models/weights-improvement-07-0.37.keras

```
1575/1575 ----- 155s 75ms/step - accuracy:
0.5663 - loss: 1.2804 - val_accuracy: 0.3703 - val_loss: 3.2461
Epoch 8/20
1575/1575 ----- 0s 49ms/step - accuracy:
0.6985 - loss: 0.8471
Epoch 8: val_accuracy improved from 0.37032 to 0.43597, saving model to
/content/saved_models/weights-improvement-08-0.44.keras
1575/1575 ----- 127s 65ms/step - accuracy:
0.6985 - loss: 0.8471 - val_accuracy: 0.4360 - val_loss: 2.7704
Epoch 9/20
1574/1575 ----- 0s 48ms/step - accuracy:
0.7622 - loss: 0.6559
Epoch 9: val_accuracy did not improve from 0.43597
1575/1575 ----- 103s 65ms/step - accuracy:
0.7622 - loss: 0.6558 - val_accuracy: 0.4358 - val_loss: 2.6851
Epoch 10/20
1575/1575 ----- 0s 49ms/step - accuracy:
0.8197 - loss: 0.5201
Epoch 10: val_accuracy improved from 0.43597 to 0.46542, saving model to
/content/saved_models/weights-improvement-10-0.47.keras
1575/1575 ----- 104s 66ms/step - accuracy:
0.8197 - loss: 0.5201 - val_accuracy: 0.4654 - val_loss: 2.8412
Epoch 11/20
1574/1575 ----- 0s 48ms/step - accuracy:
0.8615 - loss: 0.4027
Epoch 11: val_accuracy improved from 0.46542 to 0.49796, saving model to
/content/saved_models/weights-improvement-11-0.50.keras
1575/1575 ----- 140s 64ms/step - accuracy:
0.8615 - loss: 0.4027 - val_accuracy: 0.4980 - val_loss: 2.7154
Epoch 12/20
1575/1575 ----- 0s 48ms/step - accuracy:
0.8834 - loss: 0.3426
Epoch 12: val_accuracy did not improve from 0.49796
1575/1575 ----- 101s 64ms/step - accuracy:
0.8834 - loss: 0.3426 - val_accuracy: 0.4852 - val_loss: 2.7566
Epoch 13/20
1574/1575 ----- 0s 49ms/step - accuracy:
0.9104 - loss: 0.2746
Epoch 13: val_accuracy improved from 0.49796 to 0.51921, saving model to
/content/saved_models/weights-improvement-13-0.52.keras
1575/1575 ----- 103s 66ms/step - accuracy:
0.9104 - loss: 0.2746 - val_accuracy: 0.5192 - val_loss: 2.6261
Epoch 14/20
1575/1575 ----- 0s 49ms/step - accuracy:
0.9265 - loss: 0.2286
Epoch 14: val_accuracy did not improve from 0.51921
1575/1575 ----- 104s 66ms/step - accuracy:
0.9265 - loss: 0.2286 - val_accuracy: 0.5053 - val_loss: 2.7529
Epoch 15/20
1575/1575 ----- 0s 49ms/step - accuracy:
0.9439 - loss: 0.1837
Epoch 15: val_accuracy did not improve from 0.51921
1575/1575 ----- 102s 65ms/step - accuracy:
0.9439 - loss: 0.1837 - val_accuracy: 0.5162 - val_loss: 3.5029
```

```

Epoch 16/20
1575/1575 ————— 0s 48ms/step - accuracy:
0.9507 - loss: 0.1699
Epoch 16: val_accuracy improved from 0.51921 to 0.52431, saving model to
/content/saved_models/weights-improvement-16-0.52.keras
1575/1575 ————— 102s 65ms/step - accuracy:
0.9507 - loss: 0.1699 - val_accuracy: 0.5243 - val_loss: 2.6860
Epoch 17/20
1575/1575 ————— 0s 49ms/step - accuracy:
0.9594 - loss: 0.1369
Epoch 17: val_accuracy improved from 0.52431 to 0.58171, saving model to
/content/saved_models/weights-improvement-17-0.58.keras
1575/1575 ————— 144s 66ms/step - accuracy:
0.9594 - loss: 0.1369 - val_accuracy: 0.5817 - val_loss: 3.1237
Epoch 18/20
1575/1575 ————— 0s 49ms/step - accuracy:
0.9666 - loss: 0.1135
Epoch 18: val_accuracy improved from 0.58171 to 0.59769, saving model to
/content/saved_models/weights-improvement-18-0.60.keras
1575/1575 ————— 140s 65ms/step - accuracy:
0.9666 - loss: 0.1135 - val_accuracy: 0.5977 - val_loss: 2.5831
Epoch 19/20
1575/1575 ————— 0s 49ms/step - accuracy:
0.9684 - loss: 0.1165
Epoch 19: val_accuracy did not improve from 0.59769
1575/1575 ————— 141s 65ms/step - accuracy:
0.9684 - loss: 0.1165 - val_accuracy: 0.5807 - val_loss: 2.5385
Epoch 20/20
1575/1575 ————— 0s 48ms/step - accuracy:
0.9724 - loss: 0.0953
Epoch 20: val_accuracy did not improve from 0.59769
1575/1575 ————— 101s 64ms/step - accuracy:
0.9724 - loss: 0.0953 - val_accuracy: 0.5206 - val_loss: 3.5846
675/675 ————— 27s 40ms/step - accuracy:
0.5801 - loss: 2.5146
Pérdida en validación: 2.5170764923095703
Precisión en validación: 0.5775926113128662

```

```

from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
from tensorflow.keras.utils import plot_model

.flow_from_directory(
    dataset_path,
    target_size = target_size,
    batch_size = batch_size,
    color_mode = color,
    class_mode = 'categorical',

```

```

        classes = clases,
        subset='validation',
        shuffle=False # ¡IMPORTANTE PARA LA MATRIZ DE CONFUSIÓN!
    )

# =====
# 1. Curvas de Entrenamiento (Accuracy/Loss)
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Entrenamiento')
plt.plot(history.history['val_accuracy'], label='Validación')
plt.title('Precisión por Época')
plt.ylabel('Precisión')
plt.xlabel('Época')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Entrenamiento')
plt.plot(history.history['val_loss'], label='Validación')
plt.title('Pérdida por Época')
plt.ylabel('Pérdida')
plt.xlabel('Época')
plt.legend()

plt.tight_layout()
plt.savefig('training_curves.png')
plt.show()

# 2. Matriz de Confusión (¡Requiere que valid_gen tenga shuffle=False!)
valid_gen.reset() # Reiniciar el generador
y_pred = Model.predict(valid_gen, verbose=1)
y_pred_classes = np.argmax(y_pred, axis=1)

# Obtener las etiquetas reales desde el generador
y_true_classes = valid_gen.labels

cm = confusion_matrix(y_true_classes, y_pred_classes)

plt.figure(figsize=(15, 12))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=clases, yticklabels=clases) # Usar tu lista de
clases
plt.title('Matriz de Confusión')
plt.xlabel('Predicciones')

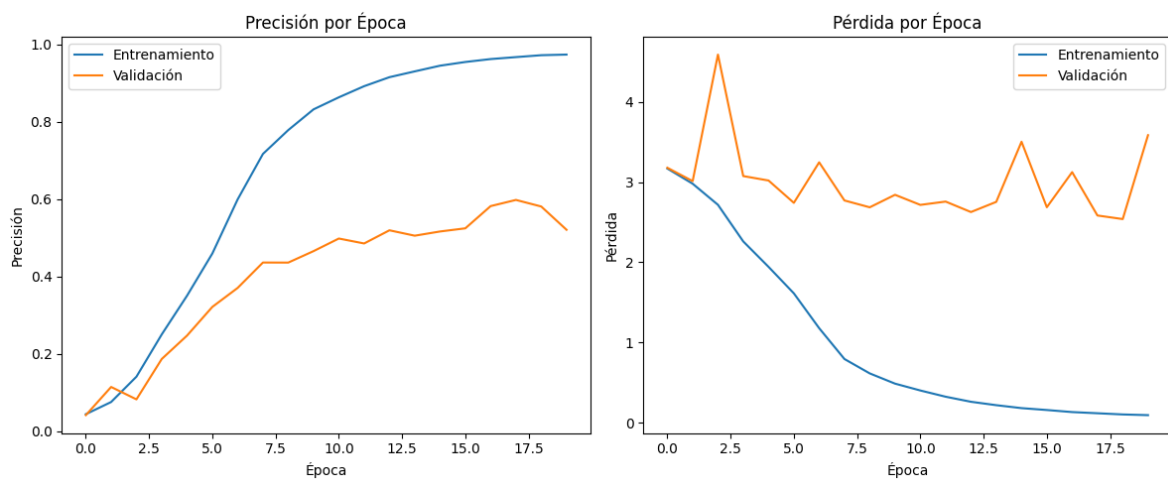
```

```

plt.ylabel('Reales')
plt.savefig('confusion_matrix.png')
plt.show()

# 4. Arquitectura del Modelo
try:
    plot_model(Model, to_file='model_architecture.png', show_shapes=True) #
    ¡Model con mayúscula!
except ImportError:
    print("Instala pydot y graphviz: pip install pydot graphviz")

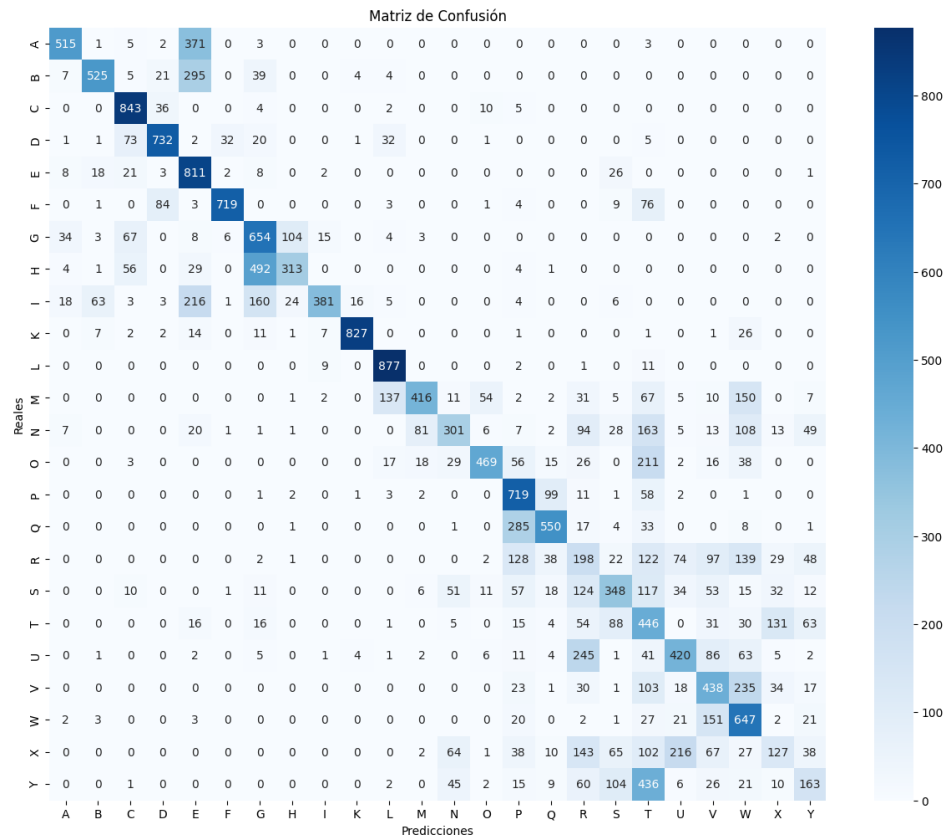
```



```

[1m675/675[0m [32m
[1m27s[0m 40ms/step
[0m[37m[0m

```



```
# 3. Ejemplos de Predicciones (MUESTRA VARIAS CLASES)
valid_gen.reset() # Reiniciar el generador

# Obtener TODAS las imágenes y etiquetas del conjunto de validación
num_samples = valid_gen.samples # Número total de muestras en validación
all_images = []
all_true_labels = []

# Iterar sobre todo el generador de validación
for _ in tqdm(range(len(valid_gen)), desc="Cargando datos de validación"):
    x_batch, y_batch = next(valid_gen)
    all_images.append(x_batch)
    all_true_labels.append(y_batch)

# Concatenar todos los batches
all_images = np.concatenate(all_images, axis=0)
all_true_labels = np.concatenate(all_true_labels, axis=0)

# Obtener 5 índices aleatorios de diferentes clases
unique_classes = np.unique(np.argmax(all_true_labels, axis=1))
selected_indices = []
for cls in np.random.choice(unique_classes, size=5, replace=False): # 5
    clases únicas
```

```

indices = np.where(np.argmax(all_true_labels, axis=1) == cls)[0]
selected_indices.append(np.random.choice(indices))

# Generar predicciones para los índices seleccionados
selected_images = all_images[selected_indices]
selected_true = np.argmax(all_true_labels[selected_indices], axis=1)
selected_pred = np.argmax(Model.predict(selected_images, verbose=0), axis=1)

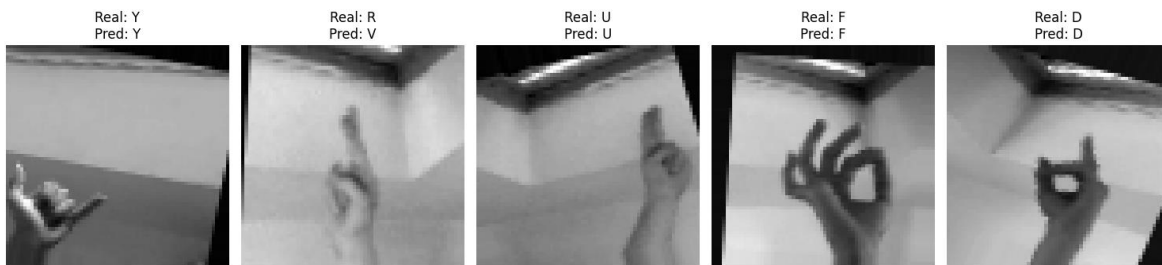
# Obtener nombres de las clases desde el generador
class_names = list(valid_gen.class_indices.keys()) # Esto debe ser ['A',
'B', ..., 'Z']

# Graficar
plt.figure(figsize=(15, 7))
for i, (idx, true_label, pred_label) in enumerate(zip(selected_indices,
selected_true, selected_pred)):
    plt.subplot(1, 5, i+1)
    plt.imshow(all_images[idx].squeeze(), cmap='gray')
    plt.title(f'Real: {class_names[true_label]}\nPred:
{class_names[pred_label]}')
    plt.axis('off')

plt.tight_layout()
plt.savefig('predictions_examples.png')
plt.show()

```

Cargando datos de validación: 100% | 675/675 [00:22<00:00, 30.55it/s]



```

from tensorflow.keras.utils import plot_model
import matplotlib.pyplot as plt

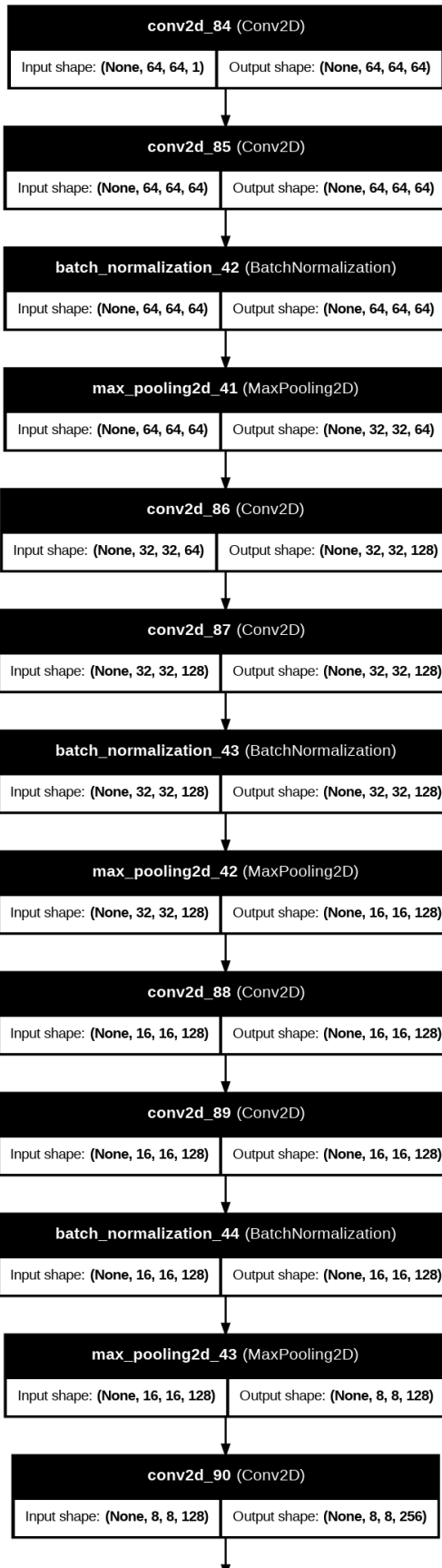
try:
    # Generar y guardar el diagrama
    plot_model(
        Model, # Tu modelo (¡con mayúscula!)
        to_file='arquitectura_red_neuronal.png',
        show_shapes=True, # Muestra las formas de entrada/salida

```

```
        show_layer_names=True, # Muestra nombres de capas
        dpi=96,                # Resolución de la imagen
        expand_nested=True     # Detalla capas dentro de bloques (ej.
Secuenciales)
    )

    # Mostrar la imagen en el notebook o entorno local
    img = plt.imread('arquitectura_red_neuronal.png')
    plt.figure(figsize=(20, 15))
    plt.imshow(img)
    plt.axis('off')
    plt.show()

except ImportError:
    print("Instala las dependencias: pip install pydot graphviz")
```





```
import tensorflow as tf

model = tf.keras.models.load_model("senas_modelo.keras")
# Para modelos secuenciales
visualkeras.layered_view(
    model,
    legend=True, # mostrar leyenda
    to_file='visualkeras.png',
    spacing=30 # espacio entre capas
)
```

